# "FUNDAMENTALS OF THE ANALYSIS OF ALGORITHM EFFICIENCY, BRUTE FORCE APPROACHES" - Q and A

BIET College, Dept. of AIML

April 18, 2026

## Contents



 

 

# 1 Question: Define algorithm. Explain asymptotic notations Big oh, Big omega and Big theta notations. (08 Marks)

## Answer

### Definition of an Algorithm

An **algorithm** is defined as a sequence of **unambiguous instructions** for solving a problem, specifically for obtaining a required output for any legitimate input in a **finite amount of time**. Beyond being a mere answer, an algorithm is a precisely defined procedure for arriving at an answer.

### Asymptotic Notations

Asymptotic notations serve as the primary language for discussing and ranking the **orders of growth** of an algorithm's efficiency, typically measured by its basic operation count.

**1. Big-oh Notation ($O$)**

- *Informal Definition:* $O(g(n))$ is the set of all functions with a **lower or same order of growth** as $g(n)$, to within a constant multiple, as $n$ goes to infinity. It effectively provides an **upper bound** on the growth of a function.
- *Formal Definition:* A function $t(n)$ is in $O(g(n))$ if there exist a positive constant $c$ and a nonnegative integer $n_0$ such that $t(n) \leq c\,g(n)$ for all $n \geq n_0$.
- *Example:* $100n + 5 \in O(n^2)$ because $100n + 5 \leq 100n + n$ (for all $n \geq 5$), which equals $101n \leq 101n^2$.

**2. Big-omega Notation ($\Omega$)**

- *Informal Definition:* $\Omega(g(n))$ represents the set of all functions with a **higher or same order of growth** as $g(n)$. This notation provides a **lower bound** for the function's growth.
- *Formal Definition:* A function $t(n)$ is in $\Omega(g(n))$ if there exist a positive constant $c$ and a nonnegative integer $n_0$ such that $t(n) \geq c\,g(n)$ for all $n \geq n_0$.
- *Example:* $n^3 \in \Omega(n^2)$ because $n^3 \geq n^2$ for all $n \geq 0$ (where $c = 1$ and $n_0 = 0$).

**3. Big-theta Notation ($\Theta$)**

- *Informal Definition:* $\Theta(g(n))$ is the set of all functions that have the **same order of growth** as $g(n)$. It provides a **tight bound**, as it bounds the function from both above and below.
- *Formal Definition:* A function $t(n)$ is in $\Theta(g(n))$ if there exist positive constants $c_1$ and $c_2$ and a nonnegative integer $n_0$ such that $c_2\,g(n) \leq t(n) \leq c_1\,g(n)$ for all $n \geq n_0$.
- *Example:* $\frac{1}{2}n(n-1) \in \Theta(n^2)$ because it is bounded above by $\frac{1}{2}n^2$ and below by $\frac{1}{4}n^2$ for all $n \geq 2$.

 

# 2 Question: Explain the general plan for analyzing the efficiency of a recursive algorithm. Suggest a recursive algorithm to find the factorial of a number. Derive its efficiency. (08 Marks)

## Part 1: General Plan for Analyzing Recursive Algorithm Efficiency (4 Marks)

Analyzing a recursive algorithm requires a different approach than iterative analysis because the cost is expressed in terms of smaller instances of the same problem. The standard five-step plan is:

| Step | Action |
|---|---|
| 1 | Determine Input Size |
| 2 | Identify Basic Operation |
| 3 | Check for Input Variability |
| 4 | Set Up Recurrence Relation |
| 5 | Solve the Recurrence |

Table 1: Five-step plan for analyzing recursive algorithms.

**Key questions for each step:**

- **Step 1:** What parameter defines the size of the problem? (e.g., $n$ for factorial, array length for sorting)
- **Step 2:** What is the fundamental unit of work? (e.g., multiplication, comparison, addition)
- **Step 3:** Does the basic operation count depend only on input size, or also on input arrangement? If yes, analyze Best, Average, and Worst cases separately.
- **Step 4:** Express the cost for size $n$ in terms of cost for size $n-1$, $n/2$, etc. Crucially, include the initial condition (base case cost).
- **Step 5:** Use backward substitution, Master Theorem, or characteristic equations to find the closed-form order of growth (e.g., $\Theta(n)$, $\Theta(n \log n)$).

## Part 2: Recursive Factorial Algorithm (2 Marks)

**Algorithm Definition:** The factorial of a non-negative integer $n$ is defined as:

- $n! = n \times (n-1)!$ for $n > 0$
- $0! = 1$ (Base Case)

**Pseudocode:**

 

**Algorithm 1** Factorial($n$)

```
// Input: A non-negative integer n
// Output: The value of n!
if n = 0 then
    return 1
else
    return Factorial(n − 1) ∗ n
end if
```

## Part 3: Efficiency Derivation (2 Marks)

Applying the General Plan from Part 1 to the Factorial algorithm:

1. **Input Size Parameter:** $n$ (the integer whose factorial is computed).
2. **Basic Operation:** Multiplication. The algorithm's time complexity is directly proportional to the number of $*$ operations performed. (Note: Function call overhead is ignored for standard time efficiency analysis.)
3. **Input Variability:** The number of multiplications depends only on the value of $n$. There is no best, average, or worst case distinction; the cost is identical for all inputs of size $n$.
4. **Recurrence Relation and Initial Condition:** Let $M(n)$ be the number of multiplications required to compute Factorial($n$).
   - When $n = 0$: The algorithm hits the base case and performs zero multiplications.
     $$M(0) = 0$$
   - When $n > 0$: The algorithm calls Factorial($n - 1$) (which costs $M(n - 1)$ multiplications) and then performs one additional multiplication (multiplying the result by $n$).
     $$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$
5. **Solve the Recurrence (Backward Substitution):**

$$\begin{aligned} M(n) &= M(n-1) + 1 \\ &= [M(n-2) + 1] + 1 = M(n-2) + 2 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3 \\ &\vdots \\ &= M(n-i) + i \end{aligned}$$

   To reach the base case $M(0)$, we substitute $i = n$:

$$\begin{aligned} M(n) &= M(n-n) + n \\ &= M(0) + n \\ &= 0 + n \\ &= n \end{aligned}$$

 

**Final Efficiency Class:** The number of multiplications executed is exactly $n$. Therefore, the time efficiency of the recursive factorial algorithm is linear:

$$M(n) \in \Theta(n)$$

 

# 3 Question: if $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then their sum $t_1(n) + t_2(n)$ belongs to the class of the larger of the two functions, specifically $O(\max\{g_1(n), g_2(n)\})$ (04 Marks)

The provided theorem states that if $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then their sum $t_1(n) + t_2(n)$ belongs to the class of the larger of the two functions, specifically $O(\max\{g_1(n), g_2(n)\})$.

## Proof of the Theorem

The proof relies on the formal definition of Big-oh notation and a basic fact about real numbers: if $a_1 \leq b_1$ and $a_2 \leq b_2$, then $a_1 + a_2 \leq 2\max\{b_1, b_2\}$.

1. **Definitions:** Since $t_1(n) \in O(g_1(n))$, there exist a positive constant $c_1$ and a nonnegative integer $n_1$ such that

$$t_1(n) \leq c_1 g_1(n) \quad \text{for all } n \geq n_1.$$

   Similarly, there exist $c_2$ and $n_2$ such that

$$t_2(n) \leq c_2 g_2(n) \quad \text{for all } n \geq n_2.$$

2. **Combining Constants:** Let $c_3 = \max\{c_1, c_2\}$ and consider $n \geq \max\{n_1, n_2\}$ so that both inequalities hold simultaneously.

3. **Derivation:**

$$\begin{aligned} t_1(n) + t_2(n) &\leq c_1 g_1(n) + c_2 g_2(n) \\ &\leq c_3 g_1(n) + c_3 g_2(n) = c_3\big[g_1(n) + g_2(n)\big] \\ &\leq c_3 \cdot 2\max\{g_1(n), g_2(n)\}. \end{aligned}$$

4. **Conclusion:** This satisfies the $O$-notation definition with the constant $c = 2c_3$ and the threshold $n_0 = \max\{n_1, n_2\}$.

## Implication for Algorithm Analysis

This property is highly significant for analyzing algorithms composed of two consecutively executed parts. It implies that the **overall efficiency of the algorithm is determined by its least efficient part**—the part with the higher order of growth.

**Example:** Consider a two-part algorithm that first checks if an array has equal elements:

- **Part 1:** Sort the array using an algorithm with efficiency $O(n^2)$.
- **Part 2:** Scan the sorted array for consecutive equal elements, which takes $O(n)$ time.
- **Total Efficiency:** The efficiency for the entire algorithm is $O(\max\{n^2, n\})$, which simplifies to $O(n^2)$.

 

# 4 Question: With a neat diagram explain different steps in designing and analyzing algorithm. (08 Marks)

## Diagrammatic Representation of the Process

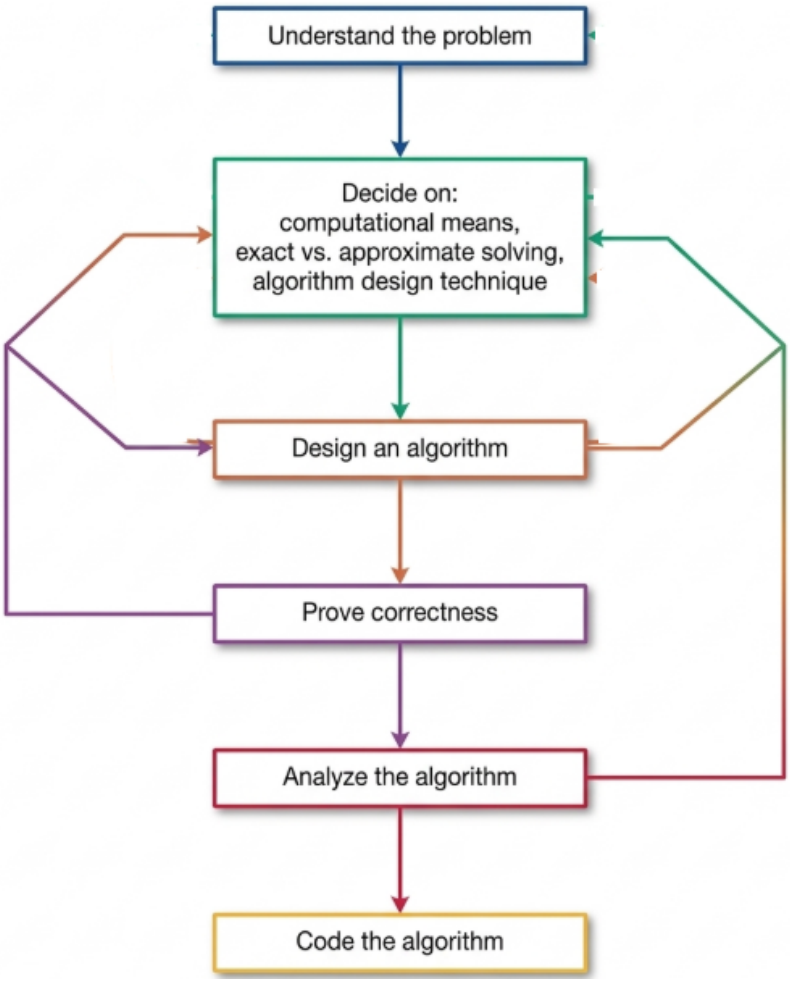


Figure 1: Algorithm design and analysis process.

## Steps in Designing and Analyzing an Algorithm

1. **Understanding the Problem**
   The first essential step is to **understand the problem completely** by reading the description carefully, working through small examples, and identifying special cases. It is crucial to define the set of **legitimate inputs (instances)** the algorithm must handle to ensure it works correctly for all boundary values.

2. **Ascertaining Capabilities of the Computational Device**
   The designer must determine the target architecture. Most algorithms are designed for the **Random-access machine (RAM)** model, where instructions are executed one after another (**sequential algorithms**). If the device can execute operations concurrently, the designer may choose **parallel algorithms**.

 

3. **Choosing between Exact and Approximate Solving**
   A decision must be made whether to solve the problem exactly or approximately. **Approximation algorithms** are often used when a problem cannot be solved exactly (e.g., square roots) or when exact solutions are unacceptably slow due to the problem's complexity.

4. **Deciding on Algorithm Design Techniques**
   A **design technique** is a general paradigm (like brute force, divide-and-conquer, etc.) applicable to various problems. These techniques provide a powerful set of tools to categorize and solve new problems systematically.

5. **Designing the Algorithm and Data Structures**
   This step requires combining design techniques with appropriate **data structures**. Choosing the right structure (e.g., an array vs. a linked list) is critical for both the efficiency and the correct functioning of the algorithm.

6. **Methods of Specifying an Algorithm**
   Once designed, the algorithm must be expressed clearly. Common methods include:

   - **Natural Language:** Easy to read but can be inherently ambiguous.
   - **Pseudocode:** A mixture of natural language and programming constructs, offering more precision and succinctness.
   - **Flowcharts:** A visual method using geometric shapes, though less common today for complex algorithms.

7. **Proving an Algorithm's Correctness**
   One must prove that the algorithm yields the **required result for every legitimate input** in a finite amount of time. **Mathematical induction** is a common technique used for these proofs because of the iterative nature of most algorithms.

8. **Analyzing the Algorithm**
   After correctness, the algorithm is evaluated based on:

   - **Efficiency:** This includes **time efficiency** (speed) and **space efficiency** (memory usage).
   - **Simplicity:** Simpler algorithms are easier to program and contain fewer bugs.
   - **Generality:** This refers to the range of inputs and the breadth of the problem the algorithm can solve.

   If the analysis is unsatisfactory, the designer returns to the drawing board to redesign.

9. **Coding the Algorithm**
   Finally, the algorithm is implemented as a **computer program**. This stage involves careful coding to maintain efficiency and rigorous **testing and debugging** to ensure validity in a practical environment.

 

# 5 Question: Write an algorithm to find the max element in an array of n elements. Give the mathematical analysis of this non-recursive algorithm. (08 Marks)

## Part 1: Algorithm to Find Maximum Element (3 Marks)

Goal: Find the largest value in an array of $n$ elements using a single scan (non-recursive).

**Algorithm 2** MaxElement($A[0..n-1]$)

```
// Input: An array A of n real numbers
// Output: The maximum element in A
maxval ← A[0]                    ▷ Assume first element is the largest initially
for i ← 1 to n − 1 do            ▷ Scan the rest of the array
    if A[i] > maxval then        ▷ Compare current element with running maximum
        maxval ← A[i]            ▷ Update if larger element found
    end if
end for
return maxval
```

## Part 2: Mathematical Analysis of Efficiency (5 Marks)

We analyze this non-recursive algorithm using the standard five-step plan for analyzing non-recursive algorithms.

### Step 1: Decide on Input Size Parameter

The input size is naturally represented by $n$, the total number of elements in the array $A$.

### Step 2: Identify the Basic Operation

Within the for loop, two operations occur:

- Comparison: $A[i] > \text{maxval}$
- Assignment: $\text{maxval} \leftarrow A[i]$

The comparison is executed on every iteration of the loop, whereas the assignment is conditional (only executed when a larger element is found). Therefore, the basic operation for efficiency analysis is the key comparison $A[i] > \text{maxval}$.

### Step 3: Check for Input Variability

The number of comparisons depends only on the size of the array ($n$) and not on the arrangement of the data. The for loop always iterates from $i = 1$ to $i = n - 1$ exactly. Thus, there is no need to distinguish between best, worst, or average cases; the count is identical for all inputs of size $n$.

 

**Step 4: Set Up a Summation Formula**

The algorithm performs one comparison for each iteration of the loop. The loop variable $i$ takes values from 1 to $n-1$ inclusive.

Let $C(n)$ be the total number of comparisons. The total count is the sum of the constant value 1 over this range:

$$C(n) = \sum_{i=1}^{n-1} 1$$

**Step 5: Solve the Summation and Determine Order of Growth**

The sum of the constant 1 repeated $(n-1)$ times is simply $(n-1)$. Using the standard summation rule $\sum_{i=l}^{u} 1 = u - l + 1$:

$$C(n) = (n-1) - 1 + 1 = \mathbf{n-1}$$

**Efficiency Class:** The function $C(n) = n - 1$ is a linear function of $n$. Therefore, the time efficiency of the MaxElement algorithm belongs to the linear class, denoted as:

$$C(n) \in \Theta(n)$$

## Summary Table

| **Analysis Component** | **Result** |
|---|---|
| Input Size | $n$ (Number of array elements) |
| Basic Operation | Comparison ($A[i] >$ maxval) |
| Summation Formula | $\sum_{i=1}^{n-1} 1$ |
| Total Count | $C(n) = n - 1$ |
| Efficiency Class | $\Theta(n)$ |

 

# 6 Question: With the algorithm derive the worst case efficiency for selection sort. (04 Marks)

## Selection Sort Algorithm (For Reference)

**Algorithm 3** SelectionSort($A[0..n-1]$)

```
// Sorts array by repeatedly finding the minimum element
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min] then
            min ← j
        end if
    end for
    SWAP(A[i], A[min])
end for
```

## Worst-Case Efficiency Derivation (4 Marks)

1. **Basic Operation:**
   The fundamental unit of work is the key comparison $A[j] < A[\text{min}]$ inside the innermost loop. All other operations (assignments, swaps) occur less frequently or are proportional to this count.

2. **Input Variability:**
   The number of comparisons in Selection Sort depends only on the array size $n$ and not on the arrangement of elements. The loops execute the same number of iterations regardless of whether the array is sorted, reverse-sorted, or random. Therefore:

   $$\text{Worst Case} = \text{Best Case} = \text{Average Case (for comparisons)}.$$

3. **Summation Setup:**
   The outer loop runs for $i = 0, 1, \ldots, n-2$.
   For each value of $i$, the inner loop runs for $j = i+1, i+2, \ldots, n-1$.
   The total number of comparisons $C(n)$ is:

   $$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

4. **Solving the Summation:**

   - Inner Sum: The number of terms from $j = i+1$ to $n-1$ is $(n-1)-(i+1)+1 = n-i-1$.

     $$C(n) = \sum_{i=0}^{n-2} (n-i-1)$$

- Outer Sum: This expands to $(n-1)+(n-2)+\cdots+1$. This is the sum of the first $(n-1)$ positive integers.
- Apply Formula: $\sum_{k=1}^{m} k = \frac{m(m+1)}{2}$. Here $m = n-1$.

$$C(n) = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

5. **Efficiency Class:**
   The leading term is $\frac{1}{2}n^2$, which grows quadratically with $n$. Ignoring the constant factor and lower-order terms, the worst-case efficiency class is quadratic:

$$C(n) \in \Theta(n^2)$$

 

# 7 Question: Develop an algorithm to search an element in an array using sequential search. Calculate the best case, worst case and average case efficiency of this algorithm. (10 Marks)

## Part 1: Sequential Search Algorithm (3 Marks)

Sequential search (also called linear search) scans the array from left to right, comparing each element with the search key until a match is found or the end of the array is reached.

**Algorithm 4** SequentialSearch($A[0..n-1]$, $K$)

```
// Input: An array A of n elements and a search key K
// Output: Index of the first occurrence of K in A, or −1 if not found
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
end while
if i < n then
    return i
else
    return −1
end if
```

## Part 2: Efficiency Analysis (7 Marks)

We analyze the time efficiency using the standard framework for non-recursive algorithms.

1. **Input Size Parameter:** The input size is $n$, the number of elements in the array.
2. **Basic Operation:** The fundamental unit of work is the key comparison $A[i] \neq K$ performed inside the while loop. The algorithm's running time is directly proportional to the number of such comparisons.
3. **Input Variability:** The number of comparisons depends heavily on the position of the search key (or its absence). Therefore, we must analyze Best, Worst, and Average cases separately.

### A. Best-Case Efficiency

**Scenario:** The search key $K$ is found at the very first position of the array ($A[0] = K$).

**Analysis:** The while loop condition $A[i] \neq K$ is checked exactly once. Since it is false, the loop body does not execute, and the algorithm terminates.

$$C_{\text{best}}(n) = 1$$

$$C_{\text{best}}(n) \in \Theta(1) \quad \text{(Constant time)}$$

 

### B. Worst-Case Efficiency

**Scenario:** The search key $K$ is not present in the array, or it is present only at the last position ($A[n-1] = K$).

**Analysis:** The algorithm must compare the key with every element in the array. The while loop iterates $n$ times, performing the comparison $n$ times before exiting.

$$C_{\text{worst}}(n) = n$$

$$C_{\text{worst}}(n) \in \Theta(n) \quad \text{(Linear time)}$$

### C. Average-Case Efficiency

**Assumptions for Average Case:**

- **Probability of Success:** Let $p$ be the probability that the search is successful (key is present). Consequently, the probability of an unsuccessful search is $(1-p)$.
- **Uniform Distribution:** In a successful search, the key is equally likely to be found in any of the $n$ positions. The probability of finding it at index $i$ (where $i$ ranges from 0 to $n-1$) is $p/n$.

**Derivation of $C_{\text{avg}}(n)$:**

- **Case 1: Successful Search**
  If the key is at index $i$, the algorithm makes $(i+1)$ comparisons (since the index starts at 0).
  The expected number of comparisons for a successful search is:

$$\sum_{i=0}^{n-1} \left[ (i+1) \times \frac{p}{n} \right] = \frac{p}{n} (1 + 2 + \cdots + n)$$

  Using the formula $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$:

$$= \frac{p}{n} \cdot \frac{n(n+1)}{2} = \frac{p(n+1)}{2}$$

- **Case 2: Unsuccessful Search**
  If the key is not found, the algorithm always performs $n$ comparisons. The probability of this case is $(1-p)$.

$$\text{Contribution} = n(1-p)$$

**Total Average Comparisons:**

$$C_{\text{avg}}(n) = \frac{p(n+1)}{2} + n(1-p)$$

**Special Case: $p = 1$ (Search Always Successful)**
If we assume the key is definitely in the array ($p = 1$), the average number of comparisons simplifies to:

$$C_{\text{avg}}(n) = \frac{n+1}{2} \approx \frac{n}{2}$$

 

On average, the algorithm searches half the list.

**Efficiency Class:** Regardless of the value of $p$, the expression for $C_{\text{avg}}(n)$ is a linear function of $n$.

$$C_{\text{avg}}(n) \in \Theta(n)$$

## Summary Table

| Case | Occurrence Condition | Comparisons Count | Efficiency Class |
|---|---|---|---|
| Best | Key is the first element | 1 | $\Theta(1)$ |
| Worst | Key is last element or absent | $n$ | $\Theta(n)$ |
| Average | Random distribution, probability $p$ | $\frac{p(n+1)}{2} + n(1-p)$ | $\Theta(n)$ |

 

# 8 Question: Explain asymptotic notations with examples. (10 Marks)

## Introduction to Asymptotic Notations

Asymptotic notations are mathematical tools used to describe the order of growth of an algorithm's time or space complexity as the input size $n$ approaches infinity. They allow us to ignore machine-dependent constants and implementation details, focusing instead on how the resource requirements scale with problem size.

The three principal notations are:

| Notation | Bound Type | Meaning |
|---|---|---|
| Big-oh $O$ | Upper bound | Grows no faster than |
| Big-omega $\Omega$ | Lower bound | Grows at least as fast as |
| Big-theta $\Theta$ | Tight bound | Grows exactly at the same rate as |

Table 2: Overview of asymptotic notations.

## 1. Big-oh Notation ($O$) – Upper Bound

**Informal Definition:** $O(g(n))$ is the set of functions that grow no faster than $g(n)$ (to within a constant multiple). It describes the worst-case scenario or maximum possible running time.

**Formal Definition:** A function $t(n)$ is said to be in $O(g(n))$ if there exist positive constants $c$ and $n_0$ such that:

$$t(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

**Example 1:** Prove that $100n + 5 \in O(n^2)$.

- For $n \geq 5$, we have $5 \leq n$.
- Therefore: $100n + 5 \leq 100n + n = 101n$.
- For $n \geq 1$, $101n \leq 101n^2$.
- Thus, choosing $c = 101$ and $n_0 = 5$ satisfies the definition.
- Conclusion: $100n + 5 \in O(n^2)$. (Note: It is also in $O(n)$ since it is linear, but the definition only requires an upper bound.)

**Example 2:**

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 \quad \text{for all } n \geq 0.$$

Therefore, $\frac{1}{2}n(n-1) \in O(n^2)$.

 

## 2. Big-omega Notation ($\Omega$) – Lower Bound

**Informal Definition:** $\Omega(g(n))$ is the set of functions that grow at least as fast as $g(n)$ (to within a constant multiple). It describes the best-case scenario or minimum possible running time.
**Formal Definition:** A function $t(n)$ is said to be in $\Omega(g(n))$ if there exist positive constants $c$ and $n_0$ such that:

$$t(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0$$

**Example 1:** Prove that $n^3 \in \Omega(n^2)$.

- $n^3 \geq n^2$ for all $n \geq 1$.
- Choosing $c = 1$ and $n_0 = 1$ satisfies the definition.
- Conclusion: $n^3 \in \Omega(n^2)$.

**Example 2:** For $\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n$, we can show:

- For $n \geq 2$, $\frac{1}{2}n \leq \frac{1}{4}n^2$ (since dividing by $n$ gives $1/2 \leq n/4$, true for $n \geq 2$).
- Therefore: $\frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{4}n^2 = \frac{1}{4}n^2$.
- Choosing $c = 1/4$ and $n_0 = 2$ proves $\frac{1}{2}n(n-1) \in \Omega(n^2)$.

## 3. Big-theta Notation ($\Theta$) – Tight Bound

**Informal Definition:** $\Theta(g(n))$ is the set of functions that grow at exactly the same rate as $g(n)$ (to within constant factors). It provides an asymptotically tight bound, capturing both upper and lower growth.
**Formal Definition:** A function $t(n)$ is in $\Theta(g(n))$ if there exist positive constants $c_1$, $c_2$, and $n_0$ such that:

$$c_2 \cdot g(n) \leq t(n) \leq c_1 \cdot g(n) \quad \text{for all } n \geq n_0$$

**Example:** Prove that $\frac{1}{2}n(n-1) \in \Theta(n^2)$.

- Upper bound: From the Big-oh example, $\frac{1}{2}n(n-1) \leq \frac{1}{2}n^2$ for $n \geq 0$. So $c_1 = 1/2$.
- Lower bound: From the Big-omega example, $\frac{1}{2}n(n-1) \geq \frac{1}{4}n^2$ for $n \geq 2$. So $c_2 = 1/4$.
- Conclusion: With $c_1 = 1/2$, $c_2 = 1/4$, and $n_0 = 2$, both inequalities hold. Therefore, $\frac{1}{2}n(n-1) \in \Theta(n^2)$.

## Comparing Growth Rates Using Limits

A practical method to determine the asymptotic relationship between two functions $t(n)$ and $g(n)$ is to compute the limit of their ratio as $n \to \infty$:

$$\lim_{n\to\infty} \frac{t(n)}{g(n)}$$

 

| Limit Value | Interpretation | Notation |
|---|---|---|
| 0 | $t(n)$ grows slower than $g(n)$ | $t(n) \in O(g(n))$ but not $\Theta(g(n))$ |
| $c > 0$ (constant) | $t(n)$ grows at the same rate as $g(n)$ | $t(n) \in \Theta(g(n))$ |
| $\infty$ | $t(n)$ grows faster than $g(n)$ | $t(n) \in \Omega(g(n))$ but not $\Theta(g(n))$ |

Table 3: Interpreting the limit of the ratio.

**Example:** Comparing $\log_2 n$ and $\sqrt{n}$

$$\lim_{n\to\infty} \frac{\log_2 n}{\sqrt{n}}$$

Apply L'Hôpital's rule (since both numerator and denominator approach $\infty$): Derivative of $\log_2 n = \frac{1}{n \ln 2}$, derivative of $\sqrt{n} = \frac{1}{2\sqrt{n}}$. Limit becomes:

$$\lim_{n\to\infty} \frac{2\sqrt{n}}{n \ln 2} = \lim_{n\to\infty} \frac{2}{\ln 2\sqrt{n}} = 0.$$

Conclusion: $\log_2 n \in O(\sqrt{n})$ but not $\Theta(\sqrt{n})$. Logarithmic functions grow slower than polynomial functions.

## Common Efficiency Classes (Increasing Order of Growth)

| Class | Name | Example Algorithm |
|---|---|---|
| $O(1)$ | Constant | Accessing an array element by index |
| $O(\log n)$ | Logarithmic | Binary search |
| $O(n)$ | Linear | Sequential search, finding maximum |
| $O(n \log n)$ | Linearithmic | Merge sort, Quick sort (average) |
| $O(n^2)$ | Quadratic | Selection sort, Bubble sort |
| $O(n^3)$ | Cubic | Naïve matrix multiplication |
| $O(2^n)$ | Exponential | Recursive Fibonacci, subset generation |
| $O(n!)$ | Factorial | Generating all permutations |

Table 4: Common efficiency classes.

## Summary Table

| Notation | Formal Condition | Provides | Example |
|---|---|---|---|
| $O(g(n))$ | $t(n) \leq c \cdot g(n)$ | Upper bound ($\leq$) | $100n \in O(n^2)$ |
| $\Omega(g(n))$ | $t(n) \geq c \cdot g(n)$ | Lower bound ($\geq$) | $n^3 \in \Omega(n^2)$ |
| $\Theta(g(n))$ | $c_2 g(n) \leq t(n) \leq c_1 g(n)$ | Tight bound ($=$) | $\frac{1}{2}n(n-1) \in \Theta(n^2)$ |

 

# 9 Question: Give the definition of an Algorithm and also discuss the characteristics of an Algorithm. (05 Marks)

## Definition of an Algorithm (1 Mark)

An **algorithm** is a finite sequence of well-defined, unambiguous instructions designed to solve a specific computational problem. It takes a set of valid inputs, processes them through a series of defined steps, and produces the desired output within a finite amount of time.

## Characteristics of an Algorithm (4 Marks)

For a procedure to qualify as an algorithm, it must satisfy the following essential properties:

| Characteristic | Explanation |
|---|---|
| 1. Finiteness | An algorithm must always terminate after a finite number of steps. It cannot run indefinitely or enter an infinite loop. |
| 2. Definiteness (Non-ambiguity) | Each step of the algorithm must be precisely defined and unambiguous. The actions to be carried out must be rigorously specified for each case, leaving no room for interpretation. |
| 3. Input | An algorithm has zero or more inputs that are provided to it before or during execution. These inputs are drawn from a specified set of legitimate instances. |
| 4. Output | An algorithm has one or more outputs that bear a specified relation to the inputs. It must produce at least one result. |
| 5. Effectiveness | Every operation in the algorithm must be basic enough that it can, in principle, be carried out exactly and in finite time by a person using only pen and paper. |
| 6. Correctness | The algorithm must produce the correct output for every possible valid input instance. It should not fail or generate incorrect results. |
| 7. Efficiency | A good algorithm is analyzed for its time efficiency (how fast it runs relative to input size) and space efficiency (how much extra memory it consumes). |

## Additional Desirable Characteristics

While the above are mandatory, the following are highly desirable for practical algorithm design:

- **Simplicity:** Simpler algorithms are easier to understand, implement, and debug, reducing the likelihood of errors.
- **Generality:** The algorithm should be applicable to a broad range of input values and problem variations, rather than being tailored to a single specific case.

 

# 10 Question: What are the various basic Asymptotic efficiency classes? (4 Marks)

## Basic Asymptotic Efficiency Classes

Algorithms are categorized into the following standard efficiency classes, ranked from the most efficient (slowest growth) to the least efficient (fastest growth):

| Class | Name | Typical Algorithm / Occurrence |
|---|---|---|
| $O(1)$ | Constant | Accessing an array element by index; pushing/popping from a stack. The running time does not depend on input size $n$. |
| $O(\log n)$ | Logarithmic | Binary Search. Typically results from repeatedly dividing the problem size by a constant factor. |
| $O(n)$ | Linear | Sequential Search; finding the maximum/minimum element in an unsorted array. Scans the entire input once. |
| $O(n \log n)$ | Linearithmic | Merge Sort, Quick Sort (average case). Common in divide-and-conquer algorithms that split the problem and combine results. |
| $O(n^2)$ | Quadratic | Selection Sort, Bubble Sort. Typically involves two nested loops processing the input. |
| $O(n^3)$ | Cubic | Naïve Matrix Multiplication. Involves three nested loops; common in linear algebra. |
| $O(2^n)$ | Exponential | Generating all subsets of an $n$-element set; recursive Fibonacci without memoization. |
| $O(n!)$ | Factorial | Generating all permutations of an $n$-element set (e.g., traveling salesman problem brute-force). |

**Key Principle:** For sufficiently large input sizes, an algorithm belonging to a lower (earlier) efficiency class will significantly outperform an algorithm from a higher (later) class, regardless of programming language or hardware optimizations.

 

# 11 Question: Give the Mathematical Analysis of Non recursive Matrix Multiplication Algorithm. (05 Marks)

## Algorithm: Non-Recursive Matrix Multiplication

**Algorithm 5** MatrixMultiplication($A[0..n-1][0..n-1]$, $B[0..n-1][0..n-1]$)

```
// Input: Two n × n matrices A and B
// Output: Their product C = A × B, an n × n matrix
for i ← 0 to n − 1 do
    for j ← 0 to n − 1 do
        C[i][j] ← 0
        for k ← 0 to n − 1 do
            C[i][j] ← C[i][j] + A[i][k] ∗ B[k][j]
        end for
    end for
end for
return C
```

## Mathematical Analysis

**Step 1: Input Size Parameter**
The natural measure of input size is $n$, the order (dimension) of the square matrices. The total number of elements is $n^2$, but $n$ suffices as the parameter.

**Step 2: Basic Operation**
The innermost loop contains two arithmetic operations: **multiplication** ($A[i][k]*B[k][j]$) and **addition** ($+$). Multiplication is typically chosen as the basic operation because it is the more expensive operation in terms of execution time. Counting one counts the other proportionally.

**Step 3: Input Variability**
The number of multiplications depends **only on the matrix size** $n$ and not on the actual values stored in matrices $A$ and $B$. Therefore, the best, worst, and average cases are identical.

**Step 4: Setting Up the Summation**
The three nested loops each iterate exactly $n$ times (from 0 to $n-1$). The total number of multiplications $M(n)$ is the sum of 1 for each iteration of the innermost loop:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

**Step 5: Solving the Summation**
We evaluate the sums from the innermost to the outermost:

- Innermost sum (over $k$): $\sum_{k=0}^{n-1} 1 = n$

 

- Middle sum (over $j$): $\sum_{j=0}^{n-1} n = n \cdot n = n^2$
- Outermost sum (over $i$): $\sum_{i=0}^{n-1} n^2 = n \cdot n^2 = n^3$

Thus, the total number of multiplications is:

$$M(n) = n^3$$

**Step 6: Efficiency Class**
The function $M(n) = n^3$ is a cubic function of $n$. Therefore, the time efficiency of the non-recursive matrix multiplication algorithm belongs to the **cubic class**:

$$M(n) \in \Theta(n^3)$$

## Summary Table

| **Analysis Component** | **Result** |
|---|---|
| Input Size | $n$ (matrix order) |
| Basic Operation | Multiplication ($A[i][k] * B[k][j]$) |
| Summation Formula | $\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$ |
| Total Count | $M(n) = n^3$ |
| Efficiency Class | $\Theta(n^3)$ |

 

# 12 Question: Give the general plan for analyzing Time efficiency of Recursive algorithms and also Analyze the Tower of Hanoi Recursive algorithm. (10 Marks)

## Part 1: General Plan for Analyzing Time Efficiency of Recursive Algorithms (4 Marks)

Analyzing a recursive algorithm requires a different approach than analyzing iterative ones because the cost is expressed in terms of smaller instances of the same problem. The standard **five-step plan** is as follows:

| Step | Action | Key Question |
|---|---|---|
| 1 | Determine Input Size Parameter | What parameter defines the size of the problem? (e.g., number of disks $n$, array length) |
| 2 | Identify Basic Operation | What is the most fundamental unit of work? (e.g., a disk move, a multiplication, a comparison) |
| 3 | Check for Input Variability | Does the basic operation count depend *only* on input size, or also on input values/arrangement? If yes, analyze Best, Average, and Worst cases separately. |
| 4 | Set Up Recurrence Relation | Express the cost for size $n$ as a function of the cost for smaller sizes. **Crucially, include the initial condition** (the cost for the base case). |
| 5 | Solve the Recurrence | Use mathematical techniques such as **backward substitution**, the Master Theorem, or characteristic equations to find a closed-form solution and determine the order of growth. |

## Part 2: Analysis of the Tower of Hanoi Recursive Algorithm (6 Marks)

### Algorithm Description

The Tower of Hanoi puzzle consists of three pegs (source, destination, auxiliary) and $n$ disks of different sizes stacked in decreasing order on the source peg. The goal is to move all $n$ disks to the destination peg under the following rules:

1. Only one disk may be moved at a time.
2. A larger disk may never be placed on top of a smaller disk.

**Recursive Solution Logic:** To move $n$ disks from source peg $A$ to destination peg $C$ using auxiliary peg $B$:

 

1. Recursively move the top $n-1$ disks from $A$ to $B$.
2. Move the largest disk directly from $A$ to $C$ (one move).
3. Recursively move the $n-1$ disks from $B$ to $C$.

**Step-by-Step Efficiency Analysis**

**Step 1: Input Size Parameter**
The natural input size is $n$, the number of disks to be moved.
**Step 2: Basic Operation**
The fundamental unit of work is **moving a single disk** from one peg to another. Each such move takes constant time, so the total time is proportional to the total number of disk moves.
**Step 3: Input Variability**
The number of moves depends **only** on the number of disks $n$. There is no variation based on different arrangements of disks; the algorithm performs the exact same number of moves for any valid initial configuration of size $n$.
**Step 4: Setting Up the Recurrence Relation**
Let $M(n)$ denote the total number of disk moves required to solve the Tower of Hanoi problem with $n$ disks.

- **Base Case ($n = 1$):** Moving a single disk requires exactly one move.

$$M(1) = 1$$

- **Recursive Case ($n > 1$):** According to the solution logic:
  - Move $n-1$ disks from source to auxiliary: $M(n-1)$ moves.
  - Move the largest disk from source to destination: 1 move.
  - Move $n-1$ disks from auxiliary to destination: $M(n-1)$ moves.

  Therefore, the recurrence relation is:

$$M(n) = M(n-1) + 1 + M(n-1) = 2M(n-1) + 1 \quad \text{for } n > 1$$

**Step 5: Solving the Recurrence Using Backward Substitution**
We solve the recurrence by repeatedly substituting the expression for $M(n-1)$:

1. Start with: $M(n) = 2M(n-1) + 1$
2. Substitute $M(n-1) = 2M(n-2) + 1$:

$$M(n) = 2[2M(n-2) + 1] + 1 = 2^2 M(n-2) + 2 + 1$$

3. Substitute $M(n-2) = 2M(n-3) + 1$:

$$M(n) = 2^2[2M(n-3) + 1] + 2 + 1 = 2^3 M(n-3) + 2^2 + 2 + 1$$

 

4. Observe the emerging pattern after $i$ substitutions:

$$M(n) = 2^i M(n-i) + (2^{i-1} + \cdots + 2^2 + 2 + 1)$$

The sum in parentheses is a geometric series: $2^{i-1} + \cdots + 2 + 1 = 2^i - 1$. Thus, the general pattern is:

$$M(n) = 2^i M(n-i) + 2^i - 1$$

5. To reach the base case, set $n - i = 1 \implies i = n - 1$:

$$M(n) = 2^{n-1} M(1) + 2^{n-1} - 1$$

6. Substitute the initial condition $M(1) = 1$:

$$M(n) = 2^{n-1}(1) + 2^{n-1} - 1 = 2 \cdot 2^{n-1} - 1 = 2^n - 1$$

**Step 6: Efficiency Class**
The closed-form solution for the number of moves is:

$$M(n) = 2^n - 1$$

This is an exponential function of $n$. Therefore, the time efficiency of the Tower of Hanoi recursive algorithm belongs to the **exponential class**:

$$M(n) \in \Theta(2^n)$$

## Summary Table

| **Analysis Component** | **Tower of Hanoi** |
|---|---|
| Input Size | $n$ (number of disks) |
| Basic Operation | Moving a single disk |
| Recurrence Relation | $M(n) = 2M(n-1) + 1$ |
| Initial Condition | $M(1) = 1$ |
| Solution | $M(n) = 2^n - 1$ |
| Efficiency Class | $\Theta(2^n)$ (Exponential) |

**Important Note:** This example illustrates a crucial point in algorithm design—a recursive algorithm can be extremely elegant and concise, yet hide an underlying **exponential inefficiency**. For $n = 64$ disks (the classic legend), the number of moves is $2^{64} - 1$, which would take over 500 billion years if one move were made per second.

 

# 13 Question: Algorithm Enigma Analysis (05 Marks)

## Algorithm Enigma

**Algorithm 6** Enigma($A[0..n-1, 0..n-1]$)

```
1: for i ← 0 to n − 2 do
2:     for j ← i + 1 to n − 1 do
3:         if A[i, j] ≠ A[j, i] then
4:             return false
5:         end if
6:     end for
7: end for
8: return true
```

## a. What does this algorithm compute? (1 Mark)

The algorithm checks whether a given $n \times n$ matrix $A$ is **symmetric**. A matrix is symmetric if $A[i][j] = A[j][i]$ for all valid indices $i, j$. The algorithm compares every element above the main diagonal (where $j > i$) with its corresponding element below the diagonal. If any pair does not match, it returns **false**; otherwise, it returns **true**.

## b. What is its basic operation? (1 Mark)

The **basic operation** is the **comparison** of two matrix elements:

$$A[i, j] \neq A[j, i]$$

This operation is executed inside the innermost loop and dominates the running time.

## c. How many times is the basic operation executed? (3 Marks)

The number of comparisons depends on whether the matrix is symmetric. We analyze the **worst case**, which occurs when the matrix **is symmetric** (no early exit). In this case, the algorithm completes all loop iterations.

- Outer loop: $i$ ranges from 0 to $n - 2$ (inclusive).
- Inner loop: For a fixed $i$, $j$ ranges from $i + 1$ to $n - 1$ (inclusive).

The number of iterations of the inner loop for a given $i$ is:

$$(n - 1) - (i + 1) + 1 = n - i - 1$$

The total number of comparisons $C(n)$ is the sum over all $i$:

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

 

Expanding the sum:

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

This is the sum of the first $(n-1)$ positive integers. Using the formula $\sum_{k=1}^{m} k = \frac{m(m+1)}{2}$ with $m = n-1$:

$$C(n) = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

**Answer:** The basic operation is executed $\frac{n(n-1)}{2}$ times in the worst case.

## d. What is the efficiency class of this algorithm? (1 Mark)

The worst-case number of comparisons is a quadratic function of $n$:

$$C(n) = \frac{1}{2}n^2 - \frac{1}{2}n$$

The leading term is $\frac{1}{2}n^2$, and constant factors and lower-order terms are ignored in asymptotic analysis. Therefore, the algorithm belongs to the **quadratic efficiency class**:

$$C(n) \in \Theta(n^2)$$

## e. Suggest an improvement or a better algorithm. (4 Marks)

**Can the efficiency class be improved?**

**No.** The efficiency class **cannot be improved** beyond $\Theta(n^2)$. Here is the proof of optimality:

- A symmetric $n \times n$ matrix has $\frac{n(n-1)}{2}$ pairs of elements above the main diagonal that must be compared with their counterparts below the diagonal.
- In the worst case (when the matrix is symmetric), **any** algorithm that correctly verifies symmetry **must examine every such pair at least once**. If any pair is skipped, there exists a matrix that differs only in that pair (making it non-symmetric) that the algorithm would incorrectly label as symmetric.
- Therefore, any algorithm for this problem has an information-theoretic lower bound of $\Omega(n^2)$ comparisons.
- Since the given algorithm achieves $O(n^2)$ in the worst case, it is **asymptotically optimal**.

**Are there any practical improvements?**

The algorithm is already well-designed for its task:

1. **Early Exit:** It returns **false** immediately upon finding a mismatch, avoiding unnecessary comparisons for non-symmetric matrices. This is the most significant practical optimization.
2. **Avoiding Redundancy:** It only compares $A[i][j]$ with $A[j][i]$ for $j > i$, performing exactly the minimum required number of comparisons ($\frac{n(n-1)}{2}$) and not duplicating work (e.g., it does not also compare $A[j][i]$ with $A[i][j]$ later).

**Conclusion:** No algorithm with a better asymptotic efficiency class exists. The given algorithm is optimal and includes good practical optimizations.

 

# 14 Question: Solve the recurrence relation: $M(n) = 2M(n-1)+1$, $M(1)=1$ for $n>1$. (05 Marks)

## Solution by Backward Substitution

**Step 1:** Start with the given recurrence

$$M(n) = 2M(n-1)+1 \tag{1}$$

**Step 2:** Substitute $M(n-1)$ using the same formula. Replace $n$ with $(n-1)$ in the recurrence:

$$M(n-1) = 2M(n-2)+1$$

Substitute into equation (1):

$$M(n) = 2[2M(n-2)+1]+1$$

$$M(n) = 2^2M(n-2)+2+1 \tag{2}$$

**Step 3:** Substitute $M(n-2)$:

$$M(n-2) = 2M(n-3)+1$$

Substitute into equation (2):

$$M(n) = 2^2[2M(n-3)+1]+2+1$$

$$M(n) = 2^3M(n-3)+2^2+2+1 \tag{3}$$

**Step 4:** Identify the emerging pattern. After $i$ substitutions, the general form becomes:

$$M(n) = 2^iM(n-i)+\left(2^{i-1}+2^{i-2}+\cdots+2+1\right)$$

The term in parentheses is a geometric series with sum $2^i-1$. Therefore:

$$M(n) = 2^iM(n-i)+2^i-1 \tag{4}$$

**Step 5:** Apply the initial condition. We want to reduce the argument of $M$ to the base case $n=1$. Set:

$$n-i=1 \quad \Longrightarrow \quad i=n-1$$

Substitute $i=n-1$ into equation (4):

$$M(n) = 2^{n-1}M(1)+2^{n-1}-1$$

**Step 6:** Use $M(1)=1$:

$$M(n) = 2^{n-1}\cdot 1+2^{n-1}-1$$

$$M(n) = 2\cdot 2^{n-1}-1$$

$$M(n) = 2^n-1$$

## Final Answer

$$\boxed{M(n)=2^n-1}$$

**Note:** This recurrence relation corresponds to the minimum number of moves required to solve the Tower of Hanoi puzzle with $n$ disks. The efficiency class of this solution is $\Theta(2^n)$.

 

# 15 Question: Design an algorithm for checking whether all elements in a given array are distinct or not. Derive its worst complexity. (06 Marks)

## Part 1: Algorithm Design

The **Element Uniqueness Problem** asks whether all elements in a given array of $n$ elements are distinct. A straightforward approach is to compare every pair of elements. If any two elements are equal, the array contains duplicates.

**Algorithm 7** UniqueElements($A[0..n-1]$)

```
// Input: An array A of n elements
// Output: Returns true if all elements are distinct, false otherwise
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i] = A[j] then
            return false                          ▷ Duplicate found
        end if
    end for
end for
return true                                       ▷ No duplicates found
```

## Part 2: Worst-Case Complexity Derivation

We analyze the algorithm using the standard framework for non-recursive algorithms.

**1. Input Size Parameter:**

The input size is $n$, the number of elements in the array.

**2. Basic Operation:**

The fundamental unit of work is the **comparison** of two array elements: $A[i] = A[j]$. This operation is executed inside the innermost loop.

**3. Worst-Case Scenario:**

The maximum number of comparisons occurs when the algorithm **never finds a duplicate** and must examine all possible pairs. This happens when:

- All elements are distinct, OR
- Only the very last pair compared (e.g., the last two elements) are equal.

In the worst case, the algorithm completes both loops entirely without an early exit.

**4. Summation Setup:**

- The outer loop runs for $i = 0, 1, 2, \ldots, n-2$.
- For a fixed value of $i$, the inner loop runs for $j = i+1, i+2, \ldots, n-1$.

The total number of comparisons $C_{\text{worst}}(n)$ is:

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

**5. Solving the Summation:**

- **Inner Sum:** The number of terms for a given $i$ is $(n-1)-(i+1)+1 = n-i-1$.

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2}(n-i-1)$$

- **Outer Sum:** Expand the sum:

$$C_{\text{worst}}(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

  This is the sum of the first $(n-1)$ positive integers.

- Using the formula $\sum_{k=1}^{m} k = \frac{m(m+1)}{2}$ with $m = n-1$:

$$C_{\text{worst}}(n) = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

**6. Efficiency Class:**
The leading term is $\frac{1}{2}n^2$. Ignoring the constant coefficient and lower-order terms, the worst-case time complexity of the algorithm is quadratic in $n$:

$$C_{\text{worst}}(n) \in \Theta(n^2)$$

## Summary Table

| **Analysis Component** | **Result** |
|---|---|
| Input Size | $n$ (array length) |
| Basic Operation | Comparison $A[i] = A[j]$ |
| Worst-Case Count | $\frac{n(n-1)}{2}$ |
| Efficiency Class | $\Theta(n^2)$ |

 

# 16 Question: Explain Brute Force method. Write an algorithm for selection sort method and apply it to the following list: 66, 11, 35, 55, 44, 22. Compute time efficiency for average case. (10 Marks)

## Brute Force Method

**Brute force** is a straightforward approach to problem-solving, usually based directly on the problem's statement and the definitions of the concepts involved. Often described by the phrase "Just do it!", the "force" implied refers to the computational power of the device rather than the intellectual sophistication of the algorithm.

While rarely the source of the most efficient solutions, brute force is an essential design strategy because it is applicable to a **very wide variety of problems**, yields reasonable algorithms for common tasks like searching or sorting, and serves as a theoretical yardstick for judging more complex alternatives.

## Selection Sort Algorithm

Selection sort works by scanning a list to find the smallest element and swapping it into its correct final position, repeating this process for the remainder of the list.

**Algorithm 8** SelectionSort($A[0..n-1]$)

```
// Sorts a given array by selection sort
// Input: An array A[0..n − 1] of orderable elements
// Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min] then
            min ← j
        end if
    end for
    SWAP(A[i], A[min])
end for
```

## Application to List: 66, 11, 35, 55, 44, 22

The algorithm performs $n-1$ passes (in this case, 5 passes for 6 elements). Elements to the left of the vertical bar | are in their final positions.

1. **Initial List:** 66, 11, 35, 55, 44, 22
2. **Pass 0:** Scan all 6 elements. The smallest is **11**. Swap with the first element (66).

   List: **11** | 66, 35, 55, 44, 22
3. **Pass 1:** Scan from 66. The smallest is **22**. Swap with the second element (66).

   List: 11, **22** | 35, 55, 44, 66

 

4. **Pass 2:** Scan from 35. The smallest is **35**. Swap with itself (no change).

   List: 11, 22, **35** | 55, 44, 66

5. **Pass 3:** Scan from 55. The smallest is **44**. Swap with the fourth element (55).

   List: 11, 22, 35, **44** | 55, 66

6. **Pass 4:** Scan from 55. The smallest is **55**. Swap with itself (no change).

   List: 11, 22, 35, 44, **55** | 66

**Final Sorted List:** 11, 22, 35, 44, 55, 66

## Average-Case Time Efficiency

The basic operation of selection sort is the **key comparison** $A[j] < A[\min]$.

- **Execution Count:** The number of comparisons depends **only on the size of the array** $n$ and is independent of the arrangement of the input values.
- **Summation Calculation:**

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}$$

- **Efficiency Class:** Because the number of comparisons is always $\frac{n(n-1)}{2}$ regardless of the input, the average-case efficiency is the same as the worst-case and best-case: $\Theta(n^2)$.

One notable advantage is that while the comparisons are quadratic, the number of **key swaps** is only linear, specifically $n - 1$.

 

# Question: Consider the following algorithm:

**Algorithm 9** Mystery($n$)

```
// Input: A non-negative integer n
S ← 0
for i ← 1 to n do
    S ← S + i * i
end for
return S
```

## (i) What does this algorithm compute?

The algorithm computes the **sum of the first $n$ squares**:

$$S = 1^2 + 2^2 + \cdots + n^2.$$

## (ii) What is its basic operation?

The **basic operation** is the **multiplication** $i * i$. While there is also an addition performed in the same loop, multiplication is traditionally identified as the basic operation in mathematical algorithms because it is typically more time-consuming than addition.

## (iii) How many times is the basic operation executed?

The number of times the basic operation is executed, $C(n)$, is determined by the range of the **for** loop, which runs from $i = 1$ to $n$. This can be expressed as a summation:

$$C(n) = \sum_{i=1}^{n} 1.$$

Using the standard summation formula $\sum_{i=l}^{u} 1 = u - l + 1$, we get:

$$C(n) = n - 1 + 1 = \mathbf{n}.$$

The basic operation is executed exactly $n$ times.

## (iv) What is the efficiency class of this algorithm?

Because the count of the basic operation is a linear function of $n$, the algorithm belongs to the **linear efficiency class**, denoted as:

$$\boxed{\Theta(n)}.$$

 

# 17 Question: With algorithm and a suitable example, explain how the brute force string matching algorithm works. Analyse for its complexity. (06 Marks)

## Algorithm

The brute-force string matching algorithm solves the problem of finding a **pattern** string of length $m$ within a **text** string of length $n$ (where $m \leq n$). It systematically checks every possible starting position in the text to see if the pattern matches there.

**Algorithm 10** BruteForceStringMatch($T[0..n-1]$, $P[0..m-1]$)

```
// Input: Text T of length n, Pattern P of length m
// Output: Index of first occurrence of P in T, or −1 if not found
for i ← 0 to n − m do
    j ← 0
    while j < m and P[j] = T[i + j] do
        j ← j + 1
    end while
    if j = m then
        return i                          ▷ Match found starting at index i
    end if
end for
return −1                                 ▷ No match found
```

## Explanation with a Suitable Example

**Text ($T$):** `"NOBODY_NOTICED_HIM"` (Length $n = 19$)
**Pattern ($P$):** `"NOT"` (Length $m = 3$)

The algorithm slides the pattern along the text, one position at a time, and compares characters from left to right.

| Attempt | Starting Index ($i$) | Aligned Substring | Comparison Result |
|---|---|---|---|
| 1 | 0 | `N O B` vs `N O T` | Mismatch at 3rd char (`B` ≠ `T`) |
| 2 | 1 | `O B O` vs `N O T` | Mismatch at 1st char (`O` ≠ `N`) |
| 3 | 2 | `B O D` vs `N O T` | Mismatch at 1st char (`B` ≠ `N`) |
| 4 | 3 | `O D Y` vs `N O T` | Mismatch at 1st char (`O` ≠ `N`) |
| 5 | 4 | `D Y _` vs `N O T` | Mismatch at 1st char (`D` ≠ `N`) |
| 6 | 5 | `Y _ N` vs `N O T` | Mismatch at 1st char (`Y` ≠ `N`) |
| 7 | 6 | `_ N O` vs `N O T` | Mismatch at 1st char (`_` ≠ `N`) |
| **8** | **7** | `N O T` vs `N O T` | **All 3 characters match!** |

- **Match Found:** The pattern `"NOT"` is found starting at index **7**.
- **Termination Condition:** The outer loop runs only up to $i = n - m = 19 - 3 = 16$. Beyond this point, fewer than $m$ characters remain in the text, so a match is impossible.

 

## Complexity Analysis

**1. Input Size Parameters:**

- $n$: Length of the text $T$
- $m$: Length of the pattern $P$

**2. Basic Operation:**
The basic operation is the **character comparison** $P[j] = T[i + j]$ performed inside the `while` loop.
**3. Worst-Case Efficiency**
The worst case occurs when the algorithm must compare all $m$ characters of the pattern for **every** possible starting position before finding a mismatch (or finding a match at the very end).

- **Number of starting positions:** $n - m + 1$
- **Comparisons per alignment:** $m$
- **Total comparisons $C_{\text{worst}}(n, m)$:**

$$C_{\text{worst}}(n, m) = m \times (n - m + 1) \approx m \cdot n$$

- **Efficiency Class:** $O(n \cdot m)$

**Example of Worst-Case Input:**

- Text: `"AAAAAAAAAAAAAAAAAAAA"` (all 'A's)
- Pattern: `"AAAAB"` (many 'A's followed by a 'B')

For each alignment, the first $m - 1$ characters match, and only the last character causes a mismatch, forcing $m$ comparisons per alignment.
**4. Average-Case Efficiency**
In practical scenarios, such as searching for a word in a natural language text, most mismatches occur **very early**—often on the first character. Consequently, the inner `while` loop terminates after just one or two comparisons for the vast majority of alignments.

For random text and pattern strings over a reasonably large alphabet (e.g., English letters), the expected number of comparisons per alignment is a small constant (close to 1). Therefore, the average-case complexity is linear with respect to the text length:

$$\text{Average-case efficiency class: } \Theta(n)$$

## Summary Table

| **Analysis Aspect** | **Result** |
|---|---|
| Algorithm Approach | Align pattern at each position; compare left-to-right |
| Worst-Case Comparisons | $m \times (n - m + 1)$ |
| Worst-Case Class | $O(n \cdot m)$ |
| Average-Case Class | $\Theta(n)$ (for natural language / random text) |

 